Assignment 6.3

2303A51250

L.Indu

Batch – 18

Task Description #1: Classes (Student Class)

Scenario

You are developing a simple student information management module.

Task

• Use an AI tool (GitHub Copilot / Cursor AI / Gemini) to complete a Student class.

• The class should include attributes such as name, roll number, and branch.

• Add a method display_details() to print student information.

• Execute the code and verify the output.

• Analyze the code generated by the AI tool for correctness and clarity.

Expected Output #1

• A Python class with a constructor (__init__) and a display_details() method.

• Sample object creation and output displayed on the console.

• Brief analysis of AI-generated code.

Prompt :

#Create a Python class named Student with attributes name, roll number, and branch.

#Add a method display_details() to print the student information.

#Also create an object of the class and display the details.

Create an object of the Student class

Display the student details

```python
#Create a Python class named Student with attributes name, roll number, and branch.
#Add a method display_details() to print the student information.
#Also create an object of the class and display the details.
class Student:
    def __init__(self, name, roll_number, branch):
        self.name = name
        self.roll_number = roll_number
        self.branch = branch

    def display_details(self):
        print(f"Student Name: {self.name}")
        print(f"Roll Number: {self.roll_number}")
        print(f"Branch: {self.branch}")
# Create an object of the Student class
student1 = Student("Indu", "CSE", "Computer Science")
# Display the student details
student1.display_details()
```

Output

```
PS C:\Users\ANIL\Desktop\AI Assisant> & C:/Users/ANIL/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Users/ANIL/Desktop/AI Assisant/StudentDetails.py"
Student Name: Indu
Roll Number: CSE
Branch: Computer Science
PS C:\Users\ANIL\Desktop\AI Assisant>
```

Analysis

- I gave a simple and clear prompt to the AI tool.
- The AI generated a correct class with constructor and display method.
- The code was easy to understand and executed without errors.

Task Description #2: Loops (Multiples of a Number)

Scenario

You are writing a utility function to display multiples of a given number.

Task

• Prompt the AI tool to generate a function that prints the first 10 multiples of a given number

using a loop.

• Analyze the generated loop logic.

• Ask the AI to generate the same functionality using another controlled looping structure (e.g.,

while instead of for).

Expected Output #2

• Correct loop-based Python implementation.

• Output showing the first 10 multiples of a number.

• Comparison and analysis of different looping approaches.

Prompt :

#Generate a Python function that prints the first 10 multiples of a given number using a for loop.

Example usage:

Code:

```
multipleNumber.py > ...
1  #Generate a Python function that prints the first 10 multiples of a given number using a for l
2  def print_multiples(number):
3      for i in range(1, 11):
4          print(f"{number} x {i} = {number * i}")
5
6  # Example usage:
7  print_multiples(5)
```

Output:

```
PS C:\Users\ANIL\Desktop\AI Assisant> & C:/Users/ANIL/AppData/Local/Python/pythoncore-3.14-64/python.
exe "c:/Users/ANIL/Desktop/AI Assisant/multipleNumber.py"
5 x 1 = 5
5 x 2 = 10
5 x 3 = 15
5 x 4 = 20
5 x 5 = 25
5 x 6 = 30
5 x 7 = 35
5 x 8 = 40
5 x 9 = 45
5 x 10 = 50
PS C:\Users\ANIL\Desktop\AI Assisant>
```

Analysis:

1.I used two different prompts to generate loop-based solutions.

2.The AI correctly used a for loop and a while loop for the same task.

Task Description #3: Conditional Statements (Age Classification)

Scenario

You are building a basic classification system based on age.

Task

• Ask the AI tool to generate nested if-elif-else conditional statements to classify age groups

(e.g., child, teenager, adult, senior).

• Analyze the generated conditions and logic.

• Ask the AI to generate the same classification using alternative conditional structures (e.g.,

simplified conditions or dictionary-based logic).

Expected Output #3

• A Python function that classifies age into appropriate groups.

• Clear and correct conditional logic.

• Explanation of how the conditions work.

Prompt :

#Generate a Python

function that classifies a

person based on age using

nested if-elif-else

conditions.

#The age groups should be

child, teenager, adult, and

senior.

```python
#Generate a Python function that classifies a person based on age using nested if-elif-else co
#The age groups should be child, teenager, adult, and senior.
def classify_age(age):
    if age < 0:
        return "Invalid age"
    elif age < 13:
        return "Child"
    elif age < 20:
        return "Teenager"
    elif age < 65:
        return "Adult"
    else:
        return "Senior"
# Example usage:
age_input = int(input("Enter the age: "))
category = classify_age(age_input)
print(f"The person is classified as: {category}")

```

Output:

```
exe "c:/Users/ANIL/Desktop/AI Assisant/ageclassification.py"
Enter the age: 15
The person is classified as: Teenager
PS C:\Users\ANIL\Desktop\AI Assisant> & C:/Users/ANIL/AppData/Local/Python/pythoncore-3.14-64/python.
exe "c:/Users/ANIL/Desktop/AI Assisant/ageclassification.py"
Enter the age: 61
The person is classified as: Adult
```

Analysis:

1.I gave a clear prompt to classify age using conditions.

2.The AI generated correct if-else logic for all age groups.

3.The output matched the expected age categories.

Task Description #4: For and While Loops (Sum of First n Numbers)

Scenario

You need to calculate the sum of the first n natural numbers.

Task

• Use AI assistance to generate a sum_to_n() function using a for loop.

• Analyze the generated code.

• Ask the AI to suggest an alternative implementation using a while loop or a mathematical

formula.

Expected Output #4

• Python function to compute the sum of first n numbers.

• Correct output for sample inputs.

• Explanation and comparison of different approaches.

Prompt :

#write a Python function sum_to_n() to calculate the sum of the first n natural numbers using for loop.

Example usage:

Code :

SumofNatural.py > ...

```
#write a Python function sum_to_n() to calculate the sum of the first n natural numbers using a for loop.
def sum_to_n(n):
    total = 0
    for i in range(1, n + 1):
        total += i
    return total

# Example usage:
print(sum_to_n(5))
print(sum_to_n(10))  # This will print the sum of the first 10 natural numbers (1+2+3+4+5+6+7+8+9+10 = 55)
```

Output

```
PS C:\Users\ANIL\Desktop\AI Assisant> & C:/Users/ANIL/AppData/Local/Python/pythoncore-3.14-64/python.exe "c:/Us
Desktop/AI Assisant/SumofNatural.py"
15
55
```

Analysis:

1. I asked the AI to generate sum logic using loops.

2. The AI gave correct code using both for and while loops.

Task Description #5: Classes (Bank Account Class)

Scenario

You are designing a basic banking application.

Task

• Use AI tools to generate a Bank Account class with methods such as deposit(), withdraw(),

and check_balance().

• Analyze the AI-generated class structure and logic.

• Add meaningful comments and explain the working of the code.

Expected Output #5

• Complete Python Bank Account class.

• Demonstration of deposit and withdrawal operations with updated balance.

• Well-commented code with a clear explanation.

Prompt :

#write a Python class BankAccount with methods deposit(), withdraw(), and check_balance().

#The program should update and display the balance after each operation.

#code should be user input and should get successful message after every transactions

Code :

bankaccount.py > BankAccount

```python
#write a Python class BankAccount with methods deposit(), withdraw(), and check_balance().
#The program should update and display the balance after each operation.
#code  should be  user input and should get successful message after every transactions

class BankAccount:
    def __init__(self, initial_balance=0):
        self.balance = initial_balance

    def deposit(self, amount):
        if amount > 0:
            self.balance += amount
            print(f"Deposited: ${amount:.2f}")
            self.check_balance()
        else:
            print("Deposit amount must be positive.")

    def withdraw(self, amount):
        if amount > 0:
            if amount <= self.balance:
                self.balance -= amount
                print(f"Withdrew: ${amount:.2f}")
            else:
                print("Insufficient funds for this withdrawal.")
            self.check_balance()
        else:
            print("Withdrawal amount must be positive.")
```

bankaccount.py > BankAccount

```
5   class BankAccount:
29              print(f"Current Balance: ${self.balance:.2f}")
30  # Example usage:
31  account = BankAccount()
32  while True:
33      print("\nChoose an operation:")
34      print("1. Deposit")
35      print("2. Withdraw")
36      print("3. Check Balance")
37      print("4. Exit")
38      choice = input("Enter your choice (1-4): ")
39
40      if choice == '1':
41          amount = float(input("Enter amount to deposit: "))
42          account.deposit(amount)
43      elif choice == '2':
44          amount = float(input("Enter amount to withdraw: "))
45          account.withdraw(amount)
46      elif choice == '3':
47          account.check_balance()
48      elif choice == '4':
49          print("Exiting the program.")
50          break
51      else:
52          print("Invalid choice. Please try again.")
```

Output :

```
PS C:\Users\ANIL\Desktop\AI Assisant> & C:/Users/ANIL/AppData/Local/Python/pythoncore-3.14-64/pyth
Desktop/AI Assisant/bankaccount.py"

Choose an operation:
1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Enter your choice (1-4): 1
Enter amount to deposit: 1000
Deposited: $1000.00
Current Balance: $1000.00

Choose an operation:
1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Enter your choice (1-4): 2
Enter amount to withdraw: 250
Withdrew: $250.00
Current Balance: $750.00

Choose an operation:
1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Enter your choice (1-4): 3
Current Balance: $750.00

Choose an operation:
1. Deposit
2. Withdraw
3. Check Balance
4. Exit
Enter your choice (1-4): 4
Exiting the program.
```

Analysis:

1. I gave a clear prompt to generate a bank account class.
2. The AI generated a well-structured class with correct logic.
3.Comments made the code easy to understand.